Malaviya National Institute of Technology Jaipur
Department of Computer Science and Engineering

Monoalphabetic Substitution Cipher and its Cryptanalysis

Frequency analysis, word-pattern analysis and iterative key recovery (C++)

Course	Cryptography Laboratory (22CPP307)
Assignment	Assignment 5 (Monoalphabetic Substitution Cipher)
Group	Group 10 (Even group number)
Team members	Rishi Saini (2024UCP1566), Nandini Verma (2024UCP1667)
Instructor	Dr. Meenakshi Tripathi
Submission date	15 October 2026
Plaintext source	Katz & Lindell, Modern Cryptography - page 40 (group 10 + 30)
Language	C++ (ISO C++11), no cryptanalysis libraries

1. Objective

Implement a monoalphabetic substitution cipher, encrypt at least one page of the textbook, and recover the plaintext and the key *using only the ciphertext* through letter-frequency analysis, word-length/repeated-word analysis and letter-pattern analysis, rejecting wrong hypotheses along the way. The key space is 26! ≈ 4.03×10^{26}, far too large for brute force, but language statistics make the cipher easy to break.

2. Plaintext, key and ciphertext

Item	Value
Plaintext	2,579 characters, 2,126 letters - Katz & Lindell, page 40
Key generation	Fisher-Yates shuffle of A-Z, seeded with the group number (seed = 10) so every run is reproducible
Plain alphabet	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher alphabet	S F B G E J D M W A K X Z C Y H Q R N P V U T O I L
Ciphertext file	outputs/ciphertext.txt (case and punctuation preserved)

```
Ciphertext (first 3 lines):
S becprsx dysx yj zygerc brihpydrshmi, wc bycprsnp py bxsnnwbsx brihpydrshmi, wn pme
geuexyhzecp yj rwdyryvn gejwcwpwycn yj nebvrwpi fsneg yc pme hrebwne zspmezspwbsx
zygexn scg snnvzhpwycn pmsp vcgerxwe brihpydrshmwb bycnprvbpwycn. Wc pmwn bmshper, te
```

3. Program structure

Function	Purpose
frequency_analysis()	Counts every ciphertext letter, prints count, percentage and bar, sorts in descending order and compares with English order ETAOINSHRDLCUMWFGYPBVKJXQZ
word_frequency_analysis()	Lists one-, two- and three-letter words and the most repeated words with counts
pattern_analysis()	Computes the letter pattern of each word (THAT = 0.1.2.0) and groups cipher words by pattern

Function	Purpose
apply_substitution()	Adds a cipher→plain mapping, resolving conflicts (a plain letter can have only one cipher letter)
display_partial_plaintext()	Shows ciphertext and partially decoded text side by side with '.' for unknown letters and progress %
verify_solution()	Builds the recovered key, decrypts, **re-encrypts the recovered plaintext and checks it reproduces the ciphertext**, and compares with the real key
iterative_recovery()	Hypothesis testing loop: proposes substitutions, scores the partial plaintext, accepts or rejects, logs the decision table

4. Cryptanalysis

4.1 Letter-frequency analysis

The ciphertext letters were counted and sorted. The most frequent ciphertext letters and the English letters they most probably represent:

Rank	Cipher letter	Frequency	Likely plain letter
1	E	13.5 %	E
2	P	10.4 %	T
3	W	8.2 %	A
4	S	6.6 %	O
5	N	6.5 %	I

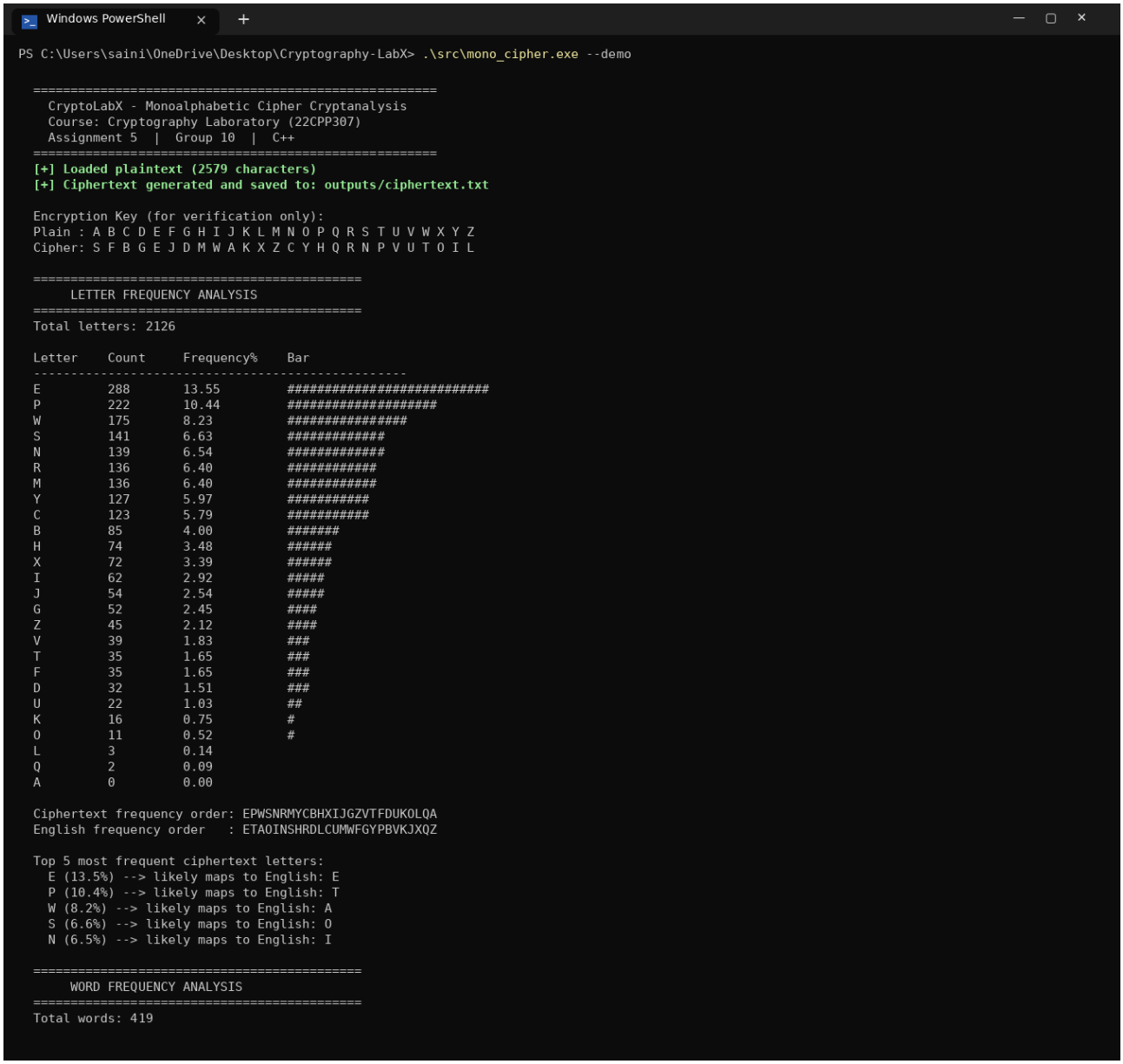


Figure 1 - Letter frequency and word frequency analysis

4.2 Word and pattern analysis

One-letter words must be **A** or **I**; the most frequent three-letter word is almost certainly **THE** (pattern 0.1.2), which fixes three letters at once; repeated words and double-letter patterns (for example 0.1.2.2.3 for LETTER-type words) narrow down the rest.

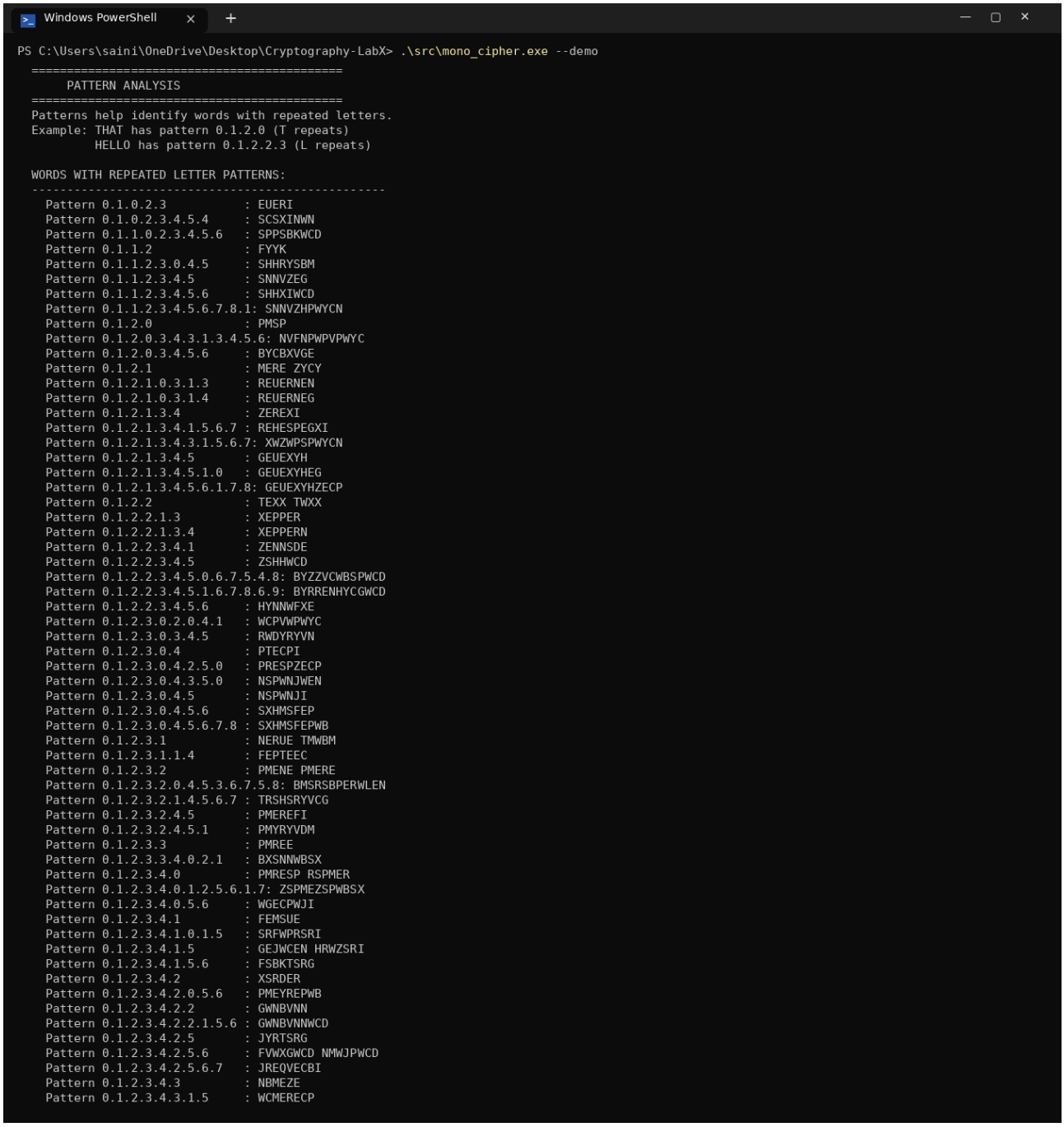


Figure 2 - Pattern analysis and the start of the decision table

4.3 Iterative recovery and decision table

Starting from the frequency-rank mapping (most frequent cipher letter → E, next → T, ...), every hypothesis is a swap of two plaintext assignments. A hypothesis is scored by the number of ciphertext letters that decode to common English words; it is **accepted only if the score improves** and is undone otherwise. The run logged 34 steps: 29 accepted and 5 rejected; in addition 1600 further swap hypotheses were tested and rejected during the final hill-climb.

Step	Observation	Possible substitution	Tested result	Decision
1	E occurs most often (288 times)	rank map to ETAOIN..	score = 10	Accepted (starting point)
2	one-letter word 'E' appears 1x	E->A, W->E	score 10 -> 5	Rejected (no gain)
3	one-letter word 'E' appears 1x	E->I, N->E	score 10 -> 5	Rejected (no gain)

Step	Observation	Possible substitution	Tested result	Decision
4	one-letter word 'F' appears 1x	F->A, W->P	score 10 -> 11	Accepted (improved)
5	one-letter word 'G' appears 1x	G->A, F->W	score 11 -> 17	Accepted (improved)
6	one-letter word 'S' appears 9x	S->A, G->O	score 17 -> 15	Rejected (no gain)
7	one-letter word 'S' appears 9x	S->I, N->O	score 17 -> 16	Rejected (no gain)
8	most frequent 3-letter word PME (44x) -> THE	M->H, Y->S	score 17 -> 153	Accepted (improved)
9	pass 1: partial words not yet in dictionary	B->F, Z->D	score 153 -> 159	Accepted (improved)
10	pass 1: partial words not yet in dictionary	C->I, N->R	score 159 -> 164	Accepted (improved)
11	pass 1: partial words not yet in dictionary	C->N, R->I	score 164 -> 170	Accepted (improved)
12	pass 1: partial words not yet in dictionary	D->Y, T->B	score 170 -> 184	Accepted (improved)
13	pass 1: partial words not yet in dictionary	F->B, T->W	score 184 -> 191	Accepted (improved)
14	pass 1: partial words not yet in dictionary	G->O, S->A	score 191 -> 227	Accepted (improved)
15	pass 1: partial words not yet in dictionary	G->P, W->O	score 227 -> 269	Accepted (improved)
16	pass 1: partial words not yet in dictionary	G->D, Z->P	score 269 -> 293	Accepted (improved)
17	pass 1: partial words not yet in dictionary	H->C, X->L	score 293 -> 297	Accepted (improved)
18	pass 1: partial words not yet in dictionary	I->M, J->U	score 297 -> 301	Accepted (improved)
19	pass 1: partial words not yet in dictionary	J->O, W->U	score 301 -> 305	Accepted (improved)
20	pass 1: partial words not yet in dictionary	N->I, R->R	score 305 -> 332	Accepted (improved)
21	pass 1: partial words not yet in dictionary	N->G, V->I	score 332 -> 333	Accepted (improved)
22	pass 1: partial words not yet in dictionary	N->U, W->G	score 333 -> 334	Accepted (improved)
23	pass 1: partial words not yet in dictionary	V->G, W->I	score 334 -> 380	Accepted (improved)
24	pass 2: partial words not yet in dictionary	B->C, H->F	score 380 -> 424	Accepted (improved)
25	pass 2: partial words not yet in dictionary	D->M, I->Y	score 424 -> 474	Accepted (improved)
26	pass 2: partial words not yet in dictionary	D->G, V->M	score 474 -> 498	Accepted (improved)
27	pass 2: partial words not yet in dictionary	H->U, N->F	score 498 -> 520	Accepted (improved)
28	pass 2: partial words not yet in dictionary	H->M, V->U	score 520 -> 528	Accepted (improved)

Step	Observation	Possible substitution	Tested result	Decision
29	pass 2: partial words not yet in dictionary	H->P, Z->M	score 528 -> 628	Accepted (improved)
30	pass 2: partial words not yet in dictionary	J->S, Y->O	score 628 -> 823	Accepted (improved)
31	pass 2: partial words not yet in dictionary	L->J, O->X	score 823 -> 867	Accepted (improved)
32	pass 3: partial words not yet in dictionary	J->F, N->S	score 867 -> 1306	Accepted (improved)
33	pass 4: partial words not yet in dictionary	A->J, L->Z	score 1306 -> 1319	Accepted (improved)
34	hill-climb converged	1600 further swap hyp	no swap improves s	Rejected (all)

5. Recovered key and validation

```
VERIFICATION
    Recovered Decryption Key (cipher -> plain):
  Cipher: A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
  Plain : J C N G E B D P Y F K Z H S X T Q R A W V U I L O M

  Plaintext accuracy : 2126/2126 letters correct (100.0%)
  Re-encryption check: encrypt(recovered plaintext, recovered key) == ciphertext ? YES [EXACT MATCH]
  Key agreement      : 25/25 key letters identical to the real key
  [+] SUCCESS! Plaintext and key fully recovered and validated.

  Actual Encryption Key Used (plain -> cipher):
  Plain : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
  Cipher: S F B G E J D M W A K X Z C Y H Q R N P V U T O I L
  Recov.: S F B G E J D M W A K X Z C Y H Q R N P V U T O I L
```

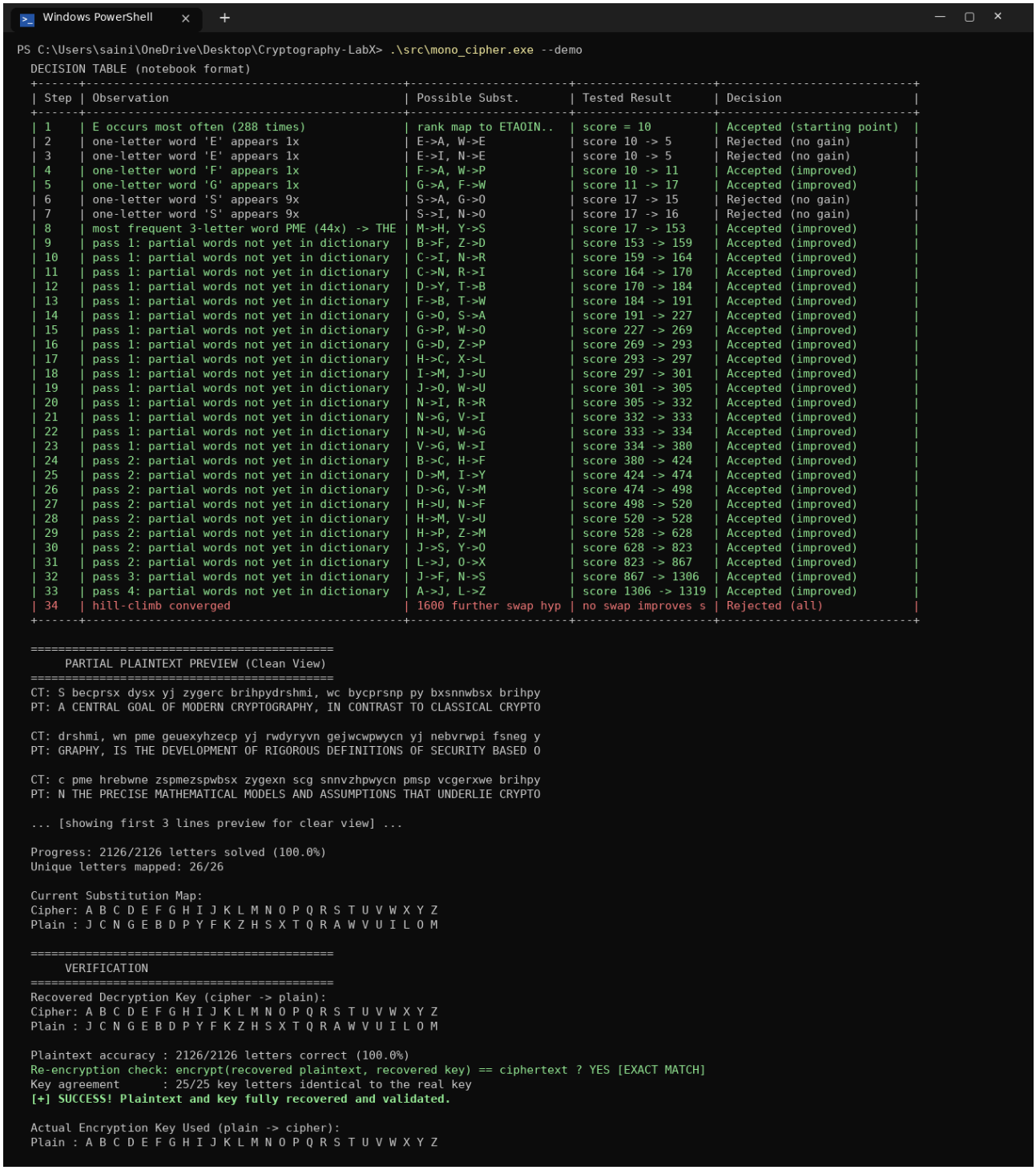


Figure 3 - Final decision table, recovered key and validation

- **Plaintext accuracy:** every letter of the 2,126 letters was recovered correctly.
- **Re-encryption check:** encrypting the recovered plaintext with the recovered key reproduces the original ciphertext exactly.
- **Key agreement:** all 25 key letters that occur in the text match the real key (the letter Q never occurs in this passage, so its mapping cannot be learned from the ciphertext).

6. Justification of the main decisions

Decision	Why it was justified
Map the most frequent cipher letter to E	E is by far the most frequent English letter; it is only a starting point and is corrected by later swaps.

Decision	Why it was justified
Treat one-letter words as A or I	These are the only common English one-letter words; the score confirmed which cipher letter is which.
Map the most frequent 3-letter word to THE	THE is the most frequent English word; the score jumped sharply when it was accepted, confirming T, H, E together.
Reject swaps that do not raise the score	Wrong hypotheses create non-words, so the partial plaintext gets worse or stays the same.
Stop when no swap improves the score	A local optimum of the word score coincided with the fully readable plaintext.

7. Conclusion

Despite a key space of about 4×10^{26}, a monoalphabetic substitution is broken with a few hundred hypothesis tests because it preserves letter frequencies, word lengths and word patterns. The iterative method recovered the complete plaintext and key, validated by re-encryption.